План тестирования

<Название проекта>.

Product version v 1.0.0

Предоставлено для:	**<Заказчик>**
Предоставил:	**<Компания-исполнитель>**
Дата:	**09 октября 2006 г.**
Версия:	**V 1.0.0**
Исполнитель:	**<Исполнитель>**

2006

История изменений

Версия	Дата	Автор	Комментарии
V1.0.0	09.10.2006	<автор>	<комментарии по текущей версии документа и внесенным изменениям>

Содержание

1 СПИСОК СОКРАЩЕНИЙ И АББРЕВИАТУР....5

2 ВВЕДЕНИЕ....6

2.1 Содержание документа....6
2.2 Цель документа....6
2.3 Целевая аудитория....6
2.4 Ссылки на проектную документацию....6
2.5 Обзор программного продукта....6
2.6 Цели тестирования....7

3 ОБЪЕМ ТЕСТИРОВАНИЯ....8

3.1 Перечень тестируемых компонентов....8
3.2 Нетестируемые компоненты....8

4 ТРЕБУЕМЫЕ РЕСУРСЫ....9

4.1 Проектная команда....9
4.2 Тестовая конфигурация....9

5 СТРАТЕГИЯ ТЕСТИРОВАНИЯ....11

5.1 Функциональное тестирование....11
5.1.1 Минимальный приемочный тест (Smoke test)....11
5.1.2 Регрессионное тестирование....11
5.1.3 Функциональный тест....11
5.2 Автоматизированное тестирование....12
5.3 Техническое тестирование....12

6 МЕТОДОЛОГИЯ ТЕСТИРОВАНИЯ....13

6.1 Обзор процесса....13
6.2 Тестовая документация....15
6.3 Доставка новых версий программного продукта....15
6.3.1 Процесс доставки....15
6.3.2 Критерии приемки на тестирование....15
6.4 Управление дефектами....15
6.4.1 Определение степени важности дефектов....16
6.4.2 Определение приоритета дефектов....16

6.4.3 Цикл жизни дефекта....16
6.4.4 Матрица переходов....17
6.4.5 Процесс взаимодействия команд тестирования и разработки ПО при работе с дефект-трекинговой систмемой....19
6.4.6 Регистрация дефектов....20
6.4.7 Критерии качества продукта....21
6.4.8 Правила наименования пользователей системы CQ....21
6.4.9 Правила ведения переписки....22

7 ГРАФИК ТЕСТИРОВАНИЯ....23

7.1 Ссылка на проектный план....23
7.2 Общая оценка временных затрат....23

8 ОЦЕНКА РИСКОВ ПРОЕКТА ТЕСТИРОВАНИЯ....24

9 ОТЧЕТЫ О РЕЗУЛЬТАТАХ ТЕСТИРОВАНИЯ....26

9.1 Журнал тестирования....26
9.2 Протокол тестирования....26
9.3 Отчет о зарегистрированных дефектах....26

1 СПИСОК СОКРАЩЕНИЙ И АББРЕВИАТУР

Сокращение / аббревиатура	Описание
Заказчик	Организация, в интересах которой разрабатывается программный продукт, имеющая полномочия утверждать требования к программному продукту и принимать результат разработок.
Дефолтное значение	Значение поля по умолчанию
Дефект	Несоответствие требованию
План тестирования	Документ, описывающий объем, методы и задачи тестирования
Продукт	Результат, произведенный в ходе выполнения проекта
Проект	Ограниченная во времени деятельность, направленная на разработку уникального продукта
Тестовый сценарий	Документ, содержащий перечень проверок для группы функционала продукта без спецификации используемых тестовых данных и обеспечивающий функциональное покрытие тестами стремящееся к 100%; может включать проверки для отдельных полей, доступных опций, графического интерфейса, целостности вводимой информации, динамического поведения полей, дефолтных значений, системные проверки, корректность вводимых значений и сохранения данных, проверки защиты от несанкционированного пользования и попыток изменения или удаления информации
Тестовый случай	Документ, содержащий детальное описание теста для одной или более функций с обозначением конкретных данных необходимых для начала тестирования и во время его, а также ожидаемого результата теста.
Набор тестовых данных	Документ, содержащий набор пререквизитов, для последующего применения их в процессах функционального и автоматизированного тестирования
МПТ	Минимальный приемочный тест
Smoke Test	МПТ

2 ВВЕДЕНИЕ

2.1 Содержание документа

Описание процесса тестирования, стратегии и подходов к тестированию комплексной системы XXXX.

Описание взаимодействия группы тестирования и команды разработчиков в процессе тестирования приложения.

2.2 Цель документа

- Спланировать управление тестированием и техническую поддержку тестирования в ходе жизненного цикла разработки XXXX;
- Обеспечение полноты тестирования в соответствии с требованиями к продукту, охват всех типов тестирования, описание необходимой проектной документации;
- Организация контроля процесса тестирования, определение методов регистрации дефектов, необходимого программного обеспечения, видов предоставления отчетов;
- Определить график работ, описать используемую методологию и стратегию тестирования;
- Определение критериев качества;
- Определение ресурсов, необходимых для реализации проекта;
- Описание документации, необходимой для обеспечения тестирования по проекту.

2.3 Целевая аудитория

Документ предназначен для последующего использования сотрудниками группы тестирования проекта и служит для понимания целей и содержания работ по тестированию программного продукта, определяет и описывает перечень работ и этапы тестирования приложения.

Также документ предназначен для ознакомления команды разработчиков с процессами взаимодействия с группой тестирования, методами регистрации и отслеживания текущего состояния дефектов, с предоставляемой по проекту тестовой документацией.

2.4 Ссылки на проектную документацию

План тестирования основан и имеет ссылки на следующую проектную документацию:
[1] XXXX.mpp (план временных затрат, сроки и даты доставки промежуточных результатов);
[2] XXXX.doc (Техническое задание);
[3] XXXX.doc (Руководство пользователей);
[4] Техническое тестирование.doc (Сценарии технического тестирования);
[5] Журнал тестирования.xls.
[6] Сценарии тестирования.xls

2.5 Обзор программного продукта

XXXX…. является интегрированным аппаратно-программным решением, предназначенным для автоматизации создания и обработки вовлечённых в деловые процессы объектов, или их электронных прототипов; для формирования правил и описания способов обработки; для организации хранения объектов обработки и обеспечения доступа к ним пользователей со своих рабочих мест; для осуществления контроля над ходом выполнения работ над объектами и их результатами.

2.6 Цели тестирования

Основными целями тестирования являются:

- Выявление проблем, связанных с несоответствием требованиям к разрабатываемому программному продукту;
- Отслеживание статуса проблем;
- Предоставление суммарной оценки качества продукта;
- Снижение рисков проекта, связанных с качеством разрабатываемого продукта.

3 ОБЪЕМ ТЕСТИРОВАНИЯ

3.1 Перечень тестируемых компонентов

Описание компанентов системы, подлежащих тестированию представлено в таблице:

Наименование, приложения/компонента	Наименование группы функций (модуля)	Функции / Ссылки / Комментарии
XXXX	**Регистрация в систему**	**Успешная регистрация в систему** **Отказ в регистрации**
	Тест обеспечения безопасности системы	**Корректное присвоение системой прав доступа на работу с функционалом в зависимости от роли пользователя:** **...**
	Главное окно приложения	**Тест GUI** **Тест главного меню** **...**
	...	**...**
	Выход из программы	**Завершение сеанса работы**

<В данной таблице приводится перечень высокоуровневых функций, для отображения общих рамок предстоящей работы, определяется набор функций высокого риска>.

Тестированию на первой итерации подлежат все функции системы как изолировано - тестирование работы функций системы внутри модулей, так и в целом - тестирование взаимодействия модулей системы, выполнение бизнес-процессов в системе, проверка полных циклов жизни документов.

Тестирование инсталляции и работы системы в различных конфигурациях.

В процессе проведения работ по тестированию на второй итерации предполагается проведение регрессионного тестирования – проверка исправления дефектов, зарегистрированных на первой итерации, тест новой функциональности, выполнение бизнес-процессов в системе, проверка полных циклов жизни документов.

Подробно перечень тестируемых компанентов будет приведен в тестовой документации, смотри пункт 6.2 Тестовая документация.

3.2 Нетестируемые компоненты

<Перечень нетестируемых компанентов>, <описание причин>.
<или>

На этапе составления плана тестирования таких функциональных компонентов определено не было.

4 ТРЕБУЕМЫЕ РЕСУРСЫ

4.1 Проектная команда

Роль	Имя и Фимилия	Место нахождения	Ответственность
Руководитель группы тестирования	XXX	г.Минск, ул. Мельникайте 2	1) Контактное лицо по проектным вопросам; 2) Написание проектной тестовой документации, разработка сценариев тестирования; 3) Разработка наборов тестовых данных для автоматизации тестирования; 3) Функциональное тестирование.
Функциональный тестировщик (xN)	XXX	г.Минск, ул. Мельникайте 2	1) Функциональное тестирование; 2) Разработка сценариев тестирования, тест-кейсов.
Технический тестировщик (xN)	XXX	г.Минск, ул. Мельникайте 2	1) Техническое тестирование: 1.1) Разработка и проведение нагрузочного тестирования; 1.2) Тестирование производительности системы; 1.3) Стресс тестирование; 1.4) Тестирование узких мест (bottle neck). 2) Автоматизация функционального тестирования: 2.1) Разработка наборов тестовых данных; 2.2) Подготовка среды для проведения автоматизированных тестов; 2.3) Разработка скриптов для автоматизации тестирования.

4.2 Тестовая конфигурация

Разработанный программный продукт должен быть установлен и протестирован на следующем оборудовании:

№	Роль	Конфигурация аппаратного обеспечения	Конфигурация программного обеспечения
1	Сервер баз данных/Агент сбора данных о загрузке компьютера при выполнении нагрузочного тестирования	Pentium 4-1600, 512RAM, 19GB HDD, 100MBit сеть	MS Windows 2000 Advanced Server SP2 Oracle 9.2.0.5 Mercury Load Runner 7.6 Agent
2	Сервер приложения (Application server)	Pentium 4-1600, 512RAM, 19GB HDD, 100MBit сеть	MS Windows 2000 Advanced Server SP2
3	Управляющий компьютер для нагрузочного тестирования/Агент для выполнения нагрузочного тестирования/Агент для выполнения автоматизированного тестирования	Pentium III-700, 512 RAM, 9GB HDD, 100MBit сеть	MS Windows 2000 Professional SP4 IE 6.0 Mercury Load Runner 7.6 Net Limiter
4	Клиент/Агент для выполнения функционального тестирования.	Pentium III-700, 512 RAM, 9GB HDD, 100MBit сеть	MS Windows 2000 Professional SP4 IE 6.0

Для конфигурации тестовой платформы будут использоваться 4 персональных компьютера – по одному на каждую из ролей. Это позволит приблизить условия тестирования к реальным и обеспечить лучшее покрытие условий тестов. Кроме того для выполнения нагрузочных тестов будет использоваться управляющий компьютер, а на остальных компьютерах тестовой платформы будут запущены агенты инструмента для проведения нагрузочных тестов. Агенты на серверах баз данных будут отвечать за сбор статистики об использовании ресурсов. Кроме того, агенты на компьютерах тестовой платформы будут задействованы для генерации требующейся загрузки.

5 СТРАТЕГИЯ ТЕСТИРОВАНИЯ

Основным методом проверки данного программного продукта будет ручное функциональное тестирование с использованием метода «черного ящика», который базируется на использовании требований и спецификаций, и не предполагает наличия каких-либо специальных знаний о конфигурации и внутренней структуре объекта испытаний.

В процессе тестирования <Название продукта> будут применяться следующие его типы:

5.1 Функциональное тестирование

Функциональное тестирование – это процесс тестирования программного продукта на предмет соответствия требованиям к программному продукту и правильности реализации всех характеристик, заложенных в продукт. Осуществляется вручную функциональным тестировщиком. Данный вид тестирования будет включать в себя два этапа:

5.1.1 Минимальный приемочный тест (Smoke test).

МПТ – это короткий тест, проверяющий основную функциональность программного продукта и его работоспособность, длящийся не более 1 часа. По результатам приемочного теста принимается решение о допуске версии программного продукта к дальнейшему тестированию.

Если очередная версия программного продукта не прошла приемочный тест, то по его результатам может быть принято решение о дальнейшем частичном тестировании программного продукта. Минимальный приемочный тест будет выполняться для каждой из поставленных версий продукта. Основная его задача – это оперативное отслеживание регрессии качества и получение высокоуровневой картины работоспособности продукта.

5.1.2 Регрессионное тестирование.

Для обеспечения надлежащего качества продукта на каждом последующем этапе будет производиться регрессионное тестирование функциональности, разработанной на предыдущей стадии и ее совместимости с новой функциональностью. Регрессионное тестирование подразумевает под собой повторное прохождение всех тестовых случаев протестированных после завершения предыдущего этапа разработки системы, проверки исправления дефектов, допущенных на этом этапе разработки системы, непоявления новых, корректное взаимодействие и совместимость функций.

5.1.3 Функциональный тест.

Функциональный тест – основной вид теста, во время которого проверяются основная функциональность программного продукта при стандартном его использовании, а так же проверяется нестандартное использование программного продукта, границы переполнения массивов данных, ввод специальных символов и т.п. Для проведения функционального теста используется весь объем созданной тестовой документации. Особое внимание уделяется тестированию бизнес-процессов системы, логических цепочек последовательностей действий пользователей, взаимодействие функций системы между собой, корректное обновление и отображение данных, логическое завершение операций.

Основная цель функционального тестирования – обеспечить максимально возможное покрытие тестами программного продукта.

5.2 Автоматизированное тестирование.

Основной целью автоматизации тестирования является решение сложных задач, выполнение которых не может быть осуществлено вручную в рамках сжатых сроков при минимальном привлечении ресурсов.

Автоматизированное тестирование подразумевает под собой использование специального программного обеспечения для проведения автоматизированных тестов на основе набора тестовых данных и написанных скриптов. Осуществляется разработчиком автоматизированных тестов.

Данный вид тестирования будет осуществляться и планироваться по дополнительному соглашению с заказчиком.

5.3 Техническое тестирование

Техническое тестирование имеет своей основной целью проверку нефункциональных требований, преъявляемых к программному продукту, будет осуществляться с использованием прикладного программного обеспечения Mercury Load Runner и производится техническим тестировщиком.

Данный вид тестирования включает в себя несколько типов тестов:

- Нагрузочное тестирование - проверка работоспособности системы при одновременной работе большого количества пользователей;
- Тест производительности - проверка быстродействия работы системы, времени отклика при вызове различных функций;
- Тестирование работоспособности системы при больших объемах данных (проверка функционирования системы при работе с крупной базой данных);
- Стресс тесты - проверка работоспособности системы в критических условиях (загрузка процессора выполнением дополнительных задач, и т.п.);
- Измерение временных параметров работы системы;
- Восстановление работоспособности системы при сбоях.

Подробное описание этапов проведения технического тестирования приведено в документе [4].

6 МЕТОДОЛОГИЯ ТЕСТИРОВАНИЯ

6.1 Обзор процесса

Подход к процессу тестирования можно отобразить в виде следующего графика (см. Рис.1):

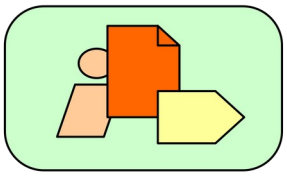


Рис. 1

Данный процесс включает в себя следующие этапы:

1. Получение требований к программному продукту (предоставляется заказчиком);
2. Анализ требований, определение целей и постановка задач по тестированию (определяется в данном документе)
3. Разработка проектной документации:
    a. план тестирования:

        i. определение тестовой стратегии;
        ii. выявление необходимых ресурсов для выполнения проекта;
        iii. определение тестовой конфигурации (необходимое оборудование и прикладное программное обеспечение);
        iv. расчет трудоемкости выполнения задач;
        v. составление предварительного графика тестирования;
    b. написание Test Cases для проведения функциональных тестов, разработка тестовых сценариев и наборов тестовых данных для осуществления технического и автоматизации функционального тестирования;
4. Установка очередной версии продукта на тестовую платформу.
5. Проведение минимального приемочного теста (МПТ) и принятие решения о приемке версии продукта для тестирования или отказе (в случае принятия решения об отказе от тестирования ожидается сборка очередной версии приложения с исправленными критическими дефектами в виду выпуска патчей либо очередной версии продукта).
6. Регрессионное тестирование - проверка и валидация исправленных дефектов. Основной целью регрессионного тестирования является проверка того, что ошибки, найденные в предыдущих версиях программного продукта исправлены, и, что при их исправлении не внесены новые.
7. Проведение полного цикла функциональных тестов; регистрация дефектов.
8. Проведение тестирования технической документации (руководства пользователей).
9. Предоставление отчета заказчику о качестве программного продукта с перечнем выявленных дефектов.
10. Исправление дефектов (осуществляется командой разработчиков/заказчиком);
11. Сборка новой версии программного продукта (осуществляется командой интеграторов, на стороне заказчика).
12. Получение группой тестирования нотификации об изменениях в новой версии продукта, анализ новой функциональности.
13. Внесение изменений в тестовую документацию в связи с имплементацией новой функциональности или изменения требований к разрабатываемому продукту (при наличии данных изменений).
14. Повторный цикл тестирования (пункт 4).
15. При выпуске кандидат-релиза, выполнение полного приемо-сдаточного тестирования.

6.2 Тестовая документация

Для обеспечения процесса тестирования требуется разработка следующей проектной тестовой документации:

- План тестирования (описание смотри в пункте 2 «Термины, сокращения, определения», данный документ), документ в формате Word;
- Сценарий тестирования_XXX (документ в формате Excel/Word);
- Техническое тестирование (сценарий проведения техничекого тестирования, документ в формате Word);
- Журнал тестирования (документ в формате Excel).

Дополнительная документация будет разрабатываться в процессе по дополнительному соглашению с заказчиком.

6.3 Доставка новых версий программного продукта

6.3.1 Процесс доставки

Новые версии продукта должны выкладываться заказчиком на FTP сервер заказчика, либо заказчик должен обеспечить возможность доставки очередной версии иным путем.

Каждая новая версия должна сопровождаться e-mail нотификацией на проектную команду с указанием:

- номера версии продукта;
- произведенных изменений.

К моменту начала проведения повторного цикла тестирования разработчики должны привести состояния дефектов в дефект-трекинговой системе в актуальное состояние (исправленные дефекты должны быть отправлены на валидацию, владельцами дефектов должны стать тестировщики, осуществившие регистрацию дефектов).

6.3.2 Критерии приемки на тестирование

Тестирование новой версии программного продукта может быть отложено полностью или частично в одном из следующих случаев:

- В новой версии программного продукта не было внесено никаких изменений;
- Программный продукт содержит ошибки, которые не позволяют провести тестирование большей части функциональности (блокированы ветви функциональности);
- Новая версия программного продукта не доступна для тестирования.

6.4 Управление дефектами

Средством регистрации и отслеживанием текущего состояния дефектов для данного проекта выбран инструмент ClearQuest от Rational Software. Выбор данного прикладного

программного обеспечения обусловлен необходимостью постоянного контроля заинтересованными сторонами появления новых дефектов, возможностью своевременного управления статусами дефектов, дистанционный доступ и отсутствие необходимости составления дополнительных отчетов, содержащих информацию о качестве тестируеммого программного продукта.

Данный продукт будет установлен локально и заказчику будет предоставлен доступ через Web.

6.4.1 Определение степени важности дефектов

Важность дефекта (Severity) требуется для правильной оценки степени влияния дефекта на корректность выполнения операций конечным пользователем.

Различают следующуе степени важности дефектов:

- «**Критический**» (Critical) – дефект, блокирующий большую часть функциональности, полностью нерабочая функция, ветвь функциональности или дефект приводящий к потере информации;
- «**Серьезный**» (Major) – дефект приводящий к некоректной работе части функциональности, недоступности опций отдельной функции, но не блокирующий тестирование в целом . Также это может быть блокирующая ошибка для отдельной функции, не оказывающей влияния на работу всей системы или ветви её функционала;
- «**Средний**» (Average) – ошибка характеризуется неправильно работающей неосновной части функциональности, не препятствующая проведению тестирования (т.е. присутствуют методы альтернативного доступа к функциям и их опциям);
- «**Незначительный**» (Minor) – приложение может успешно функционировать, не нарушена бизнес логика, исправление дефекта откладывается на последнюю очередь.

6.4.2 Определение приоритета дефектов

Приоритет дефекта определяется, чтобы правильно оценить, на сколько срочно требуется исправление дефекта.

Приоритет должен быть основан на:

- Серьезности дефекта.
- Критичности функции для пользователя.

Различают следующие приоритеты:

- «**Исправить незамедлительно**» (Resolve immediately) – исправление дефета имеет наивысший приоритет, должно быть осуществлено в первую очередь в кратчайшие сроки.
- «**Уделить повышенное внимание**» (Give high attention) – нет необходимости в сверхсрочном исправлении дефекта, но устранение ошибки должно быть осуществлено непосредственно.
- «**Обычный**» (Normal queue) – дефолтный приоритет при регистрации дефектов.
- «**Низкий**» (Low priority) – дефекты исправляются только после устранения всех других ошибок в программе.
- **Улучшение** (Enchancement) – предложения тестовой группы по улучшению качества интерфейса (внедряется только после валидации заказчиком).

6.4.3 Цикл жизни дефекта

Схематично цикл жизни дефекта можно описать с помощью следующей диаграммы (Рис.2):



Рис.2

6.4.4 Матрица переходов

Матрица переходов выглядит следующим образом (Рис.3.):

From / To	Submitted	Opened	Resolved	Closed	Duplicate	Postponed
Submitted						
Opened	Open		Reject	Re_open	Open	Open
Resolved		Resolve				
Closed	Close	Close	Validate		Close	Close
Duplicate	Duplicate	Duplicate	Duplicate	Duplicate		Duplicate
Postponed		Postpone				

Рис.3.

В процессе проведения тестирования системы XXXX предполагается использование следующей схемы взаимодействия группы тестирования с заказчиком по совместному процессу отслеживания и исправления дефектов:

№	Предыдущее состояние	Действие	Новое состояние	Кто произвел	Ответственное лицо	Комментарии
1	Нет No	Регистрация дефекта Submit	Зарегистрирован Submitted	Тестировщик / Лидер группы тестирования	Лидер группы тестирования	Должны быть заполнены все обязательные атрибуты
2	Зарегистрирован	Открыть Open	Открыт Opened	Лидер группы тестирования	Лидер команды разработчиков	Передача дефекта команде разработчиков для

№	Предыдущее состояние	Действие	Новое состояние	Кто произвел	Ответственное лицо	Комментарии
	Submitted					проведения исследования проблемы и исправления
3	Зарегистрирован Submitted	Закрыть Close	Закрыт Closed	Лидер группы	Лидер группы тестирования	Резолюция о причине закрытия дефекта
4	Зарегистрирован Submitted	Диблировать Duplicate	Дублирован Duplicated	Лидер команды тестирования	Лидер команды тестирования	Дефект дублирует ранее зарегистрированный, указать номер дублирующего дефекта
5	Открыт Opened	Закрыть Close	Закрыт Closed	Лидер команды разработчиков	Лидер команды тестирования	Резолюция о причине закрытия дефекта
6	Открыт Opened	Изменить Modify	Открыт Opened	Лидер команды разработчиков	Ответственный за исправление разработчик (Разработчик функции)	Назначение владельцем дефекта ответственного за исправление разработчика
7	Открыт Opened	Отложить Postpone	Отложен Postponed	Лидер команды разработчиков	Лидер команды разработчиков	Исправление дефекта требует проведения дополнительного исследования или требует больших временных затрат
8	Открыт Opened	Исправить Resolve	Исправлен Resolved	Разработчик функции	Тестировщик, осуществивший регистрацию	Дефект исправлен и готов к тестированию
9	Отложен Postponed	Открыть Open	Открыт Opened	Лидер команды разработчиков	Лидер команды разработчиков	Дефект снова принят в работу
10	Отложен Postponed	Закрыть Close	Закрыт Closed	Лидер команды разработчиков	Лидер команды разработчиков	Резолюция о причине закрытия дефекта
11	Исправлен Resolved	Валидировать Validate	Закрыт Closed	Тестировщик	Тестировщик	Дефект исправлен и закрыт
12	Исправлен Resolved	Отказать в валидации Reject	Открыт Opened	Тестировщик	Разработчик функции	Дефект не исправлен, указываются причины отказа в

№	Предыдущее состояние	Действие	Новое состояние	Кто произвел	Ответственное лицо	Комментарии
						валидации
13	Закрыт Closed	Открыть Re-open	Открыт Opened	Лидер группы тестирования	Лидер команды разработчиков	Дефект проявился повторно
14	Дублирован Duplicated	Открыть Open	Открыт Opened	Лидер группы тестирования	Лидер команды разработчиков	Дефект ошибочно переведен в статус «Дублирован», отправлен на исправление
15	Дублирован Duplicated	Закрыть Close	Закрыт Closed	Лидер группы тестирования	Лидер группы тестирования	Дефект дублирован и закрыт.

В качестве резолюции о причине закрытия дефекта из состояний «зарегистрирован», «открыт», «отложен» могут быть рассмотрены следующие варианты:

- Разработано согласно требованиям (дефект открыт ошибочно);
- Дефект не существенный и не требует исправления (не нарушены бизнес-требования);
- Корекция дефекта нецелесообразна - исправление дефекта окажет критическое действие на большую часть функциональности (предложение варианта по изменению требований к продукту);

6.4.5 Процесс взаимодействия команд тестирования и разработки ПО при работе с дефект-трекинговой систмемой.

- При регистрации дефекта он приобретает статус Submitted (регистрируется тестировщиками) – выставляется owner дефекта – лидер команды тестирования;
- Лидер команды тестирования проверяет воспроизведение дефекта и открывает его на лидера команды разработчиков (дефект приобретает статус Opened);
- Лидер команды разработчиков квалифицирует дефект и назначает ответственного за его исправление разработчика (action = Modify), редактируется значение только поля Owner, как правило это разработчик функции (дефект остается в состоянии Opened).
  В системе автоматически формируется и отсылается нотификация ответственному лицу – разработчику функции о том, что он назначен ответственным за исправление данного дефекта;
- Разработчик после исправления дефекта проставляет версию билда, в котором будет доступен фикс для тестирования, выбирает action = Resolve, назначается owner = тестировщик, осуществивший регистрацию дефекта, дефект переходит в состояние Resolved;
- Тестировщик проверяет исправление дефекта, в случае если дефект исправлен корректно - вызывает action = Validate, дефект переходит в состояние Closed. Если дефект не исправлен или исправлен частично, тестировщик вызывает action = Reject (указывает в

поле Notes номер версии билда, в котором дефект был оттестирован, описывает проблемы, почему дефект не исправлен), проставляется owner = разработчик, осуществлявший исправление, дефект снова переходит в состояние Opened. В этом случае разработчик снова осуществляет исправление и переводит дефект в состояние Resolved с указанием новой версии приложения в котором будет доступен фикс. Цикл повторяется.

- Из состояния Closed дефект может быть снова открыт (применяется action – Re-open) в случае его повторного проявления.

На всех промежуточных этапах дефект можно редактировать (action = Modify), это действие не изменяет статуса дефекта, но сохраняет изменения и комментарии.

Лидер команды разработчиков может закрывать дефект (action = Close) с выставлением резолюций (в случаях если дефект не обнаружен, разработано согласно требованиям, исправление дефекта невозможно – делается запрос на изменение требований заказчикам).

Исправление дефекта по разным причинам может быть отложено – action = Postpone (state = Postponed).

В случае если подобный дефект уже был зарегистрирован ему присваивается статус Duplicate с указанием номера дефекта, которого он дублирует.

Лидеры команд имеют право на удаление дефектов из системы (action = Delete) (!не рекомендуется!, информация по дефекту и его история полностью утрачивается).

В случае, если дефект не воспроизводится у разработчиков, недостаточно подробно описан процесс воспроизведения дефекта или не ясен общий смысл ошибки, лидер команды разработчиков (или разработчик, занимающийся исправлением дефекта) выставляет в дефекте Owner = тестировщик, осуществивший регистрацию дефекта и пишет письмо тестировщику с перечнем конкретных вопросов, что именно непонятно в зарегистрированном дефекте. Тестировщик дополняет дефект в CQ необходимыми комментариями, выставляет Owner = лицо, запросившее дополнительную информацию и нотифицирует почтой этого разработчика о том, что недостатки описания дефекта устранены.

6.4.6 Регистрация дефектов

Перечень необходимых для заполнения полей при регистрации дефекта приведен в следующей таблице:

Название поля	Описание	Перевод
ID	Идентификационный номер дефекта, заполняется автоматически системой	ID
Статус	Текущее состояние дефекта	State
Заголовок	Название дефекта (краткое описание – привязка дефекта к функции)	Headline
Ответственное лицо	Член команды, принимающий решение о дальнейшей разработке ошибки, определяет (или предпринимает) комплекс мер по устранению дефекта	Owner
Важность	Серьезность дефекта, определяется для правильной оценки степени влияния дефекта на корректность выполнения операций конечным пользователем	Severity

Название поля	Описание	Перевод
Приоритет	Определяется для указания разработчикам на сколько срочно требуется исправление дефекта	Priority
Модуль	Наименование модуля, в котором обнаружен дефект	Module Name
Группа	Наименование группы функций, принадлежащих модулю, в которых обнаружен дефект	Group
Функция	Функция, в которой допущен дефект	Function
Версия тестируемого программного продукта	Номер версии программного продукта	Build №
Компонент исправления	Компонент, в котором можно будет протестировантъ исправление дефекта (заполняется разработчиком, осуществившим исправление дефекта)	Fixed In
Описание	Полное описание дефекта, методы его воспроизведения, влияние на другие поля и функции	Description
Новая заметка	Поле служит для внесения дополнительных сведений по дефекту, облегчения взаимодействия между командами разработчиков и группой тестирования	New note
История заметок	Поле служит для воспроизведения истории внесения заметок	Notes log
История изменений	Данное поле описывает все действия произведенные с дефектом, кто производил, даты изменений	History
Резолюция	Резолюция, выставляемая при закрытии дефектов из промежуточных состояний, неравных Resolved.	Resolution
Присоединенные файлы	Присоединенные файлы для более подробного описания дефекта (скрин-шоты, логи ошибок)	Attachments

6.4.7 Критерии качества продукта

- Программный продукт должен работать в соответствии со спецификациями и требованиями;
- Функциональность должна полностью покрывать все требования, изложенные в руководствах пользователей (в случае разбиения развития программного продукта на итерации, должна быть полностью реализована функциональность, запланированная на итерацию);
- Программный продукт не должен иметь известных дефектов со статусом «Критический» и «Высокий» к моменту поставки программного продукта заказчику.

6.4.8 Правила наименования пользователей системы CQ

Наименования пользователей в CQ происходит по следующей схеме: краткое имя = 1-ая буква имени + 2-е первые буквы фамилии. Например, для пользователя Natallia Hustyr, краткое имя в системе будет обозначено как NHU.

6.4.9 Правила ведения переписки

В процессе проведения тестирования возникает необходимость ведения переписки с командой разработчиков. Со строны заказчика контактными лицами как правило являются менеджер проекта для решения проектных вопросов, системный аналитик для решения вопросов связанных с корректностью функционирования системы, лидер команды разработчиков для обеспечения решения вопросов, связанных с отслеживанием дефектов, зарегистрированных в системе; со стороны группы тестирования проектные вопросы обращаются к лидеру команды тестирования / менеджеру проекта со стороны исполнителя, вопросы, связанные с дефектами, решаются или с лидером группы тестирования, или непосредственно с тестировщиком, зарегистрировавшим данный дефект.

Сруктура письма обязательно должна включать в себя «Заголовок», однозначно определяющий тему письма и тело письма, содержащее непосредственно запрос. В копию письма включаются все заинтересованные лица по данному впросу.

Ответ на письмо обязательно должен быть отправлен не позднее четырех часов с момента получения письма. В случаях, когда нет возможности составить корректный ответ на поставленные в письме вопросы, получатель письма должен выслать нотификацию о получении письма и предположительную дату, когда будет выслан развернутый ответ на запрос. В случае, если ответ на письмо не получен в указанные сроки, письмо дублируется, в копию ставится менеджер проекта, который будет осуществлять контроль за исполнительностью подчиненного.

7 ГРАФИК ТЕСТИРОВАНИЯ

7.1 Ссылка на проектный план

Перечень и подробные временные затраты на выполнение каждой из поставленных задач приведены в документе Project_XXXX.mpp.

7.2 Общая оценка временных затрат

Общий объем работ по обновлению и разработке тестовой документации, проведению регрессионных тестов и проверки новой функциональности по системе XXXX оценивается в N чел/дней. Из них n чел/дней отводится на разработку и обновление тестовой документации по проекту, m чел/дней на инсталляцию и настройку приложения, X чел/дней на проведение функционального тестирования новой версии программного продукта, Y чел/дней отводится на подведение итогов тестирования, оформления отчетов о результатах тестирования, предоставление отчетов о качестве программного продукта заказчику. Общая длительность проекта составляет Z рабочих дней в связи с распределением задач на нескольких человек и с учетом того, что функциональное тестирование стартует по окончании этапа проектирования тестов. Для получения подробных комментариев по трудозатратам смотрите документ [1].

8 ОЦЕНКА РИСКОВ ПРОЕКТА ТЕСТИРОВАНИЯ

Риск	Вероятность наступления	Влияние на проект	Предварительные меры
Изменение функциональных требований на завершающем этапе проекта	Низкая	Высокое	Ответственный за риск: Заказчик. Предварительные меры: тестовая документация должна иметь адаптивный характер, предусмотреть возможность быстрого внесения изменений (новых требований по проекту) и минимизирования затрат на проведение дополнительных тестов.
Программный продукт имеет известные дефекты со статусом «Критический» и «Высокий» к моменту поставки программного продукта заказчику	Низкая	Высокое	Ответственный за риск: Команда разработчиков. Предварительные меры: своевременная нотификация группой тестирования команды разработчиков обо всех новых дефектах, идентифицированных при проведении функциональных и технических тестов.
Отставание от графика работ	Низкая	Среднее	Ответственный за риск: группа тестирования / команда разработчиков. Предварительные меры: чёткое планирование временных затрат на выполнение поставленных задач, своевременная нотификация заказчика обо всех проблемах, возникающих в процессе тестирования и имплементации исправлений по дефектам и внедрения новой функциональности.
Поставленный для проведения тестирования программный продукт имеет дефекты	Низкая	Высокое	Ответственный за риск: Команда разработчиков. Требуется оперативная реакция со стороны команды разработки для

блокирующие основную функциональность			предоставления исправлений.
Не налажен процесс взаимодействия команд разработчиков и тестирования	Низкая	Среднее	Ответственный за риск: Менеджерский состав команд разработчиков и тестирования. Требуется: предварительное согласование постановки процесса взаимодействия между командами, обсуждение условий поставки следующих версий программного продукта, предварительная высылка нотификаций, сопровождение необходимой проектной документацией
Возможность обнаружения ошибок на стороне заказчика после проведения тестирования	Низкая	Высокое	Ответственный за риск: группа тестирования. Методы предупреждения: Перед проведением тестирования производится согласование с заказчиком тестовых сценариев. При этом заказчиком производится оценка степени покрытия тестами программного приложения.

9 ОТЧЕТЫ О РЕЗУЛЬТАТАХ ТЕСТИРОВАНИЯ

Отчеты о ходе тестирования будут высылаться еженедельно (в конце рабочей недели) и включать в себя перечень активностей команды тестирования за прошедшую неделю (по соглашению с заказчиком возможно дополнительно предоставление списка артефактов для доставки на следующей неделе). Сроки доставки конечных продуктов будут оговорены в пункте 7. График тестирования.

9.1 Журнал тестирования

В процессе тестирования, результаты выполнения будут заноситься в Журнал формата Excel следующего содержания:

Название проекта				
Тестируемый модель				
Функция	**Тип теста**	**Тестовый случай**	**Pass/Fail**	**Описание**
Тестируемая функция	GUI / Case	Перечень тестовых случаев	Результат теста	Перечень номеров дефектов, комментарии по недостаткам.

Основной целью Журнала тестирования является высокоуровневое отображение качества продукта. В нем приведен общий перечень тестируемых функций без глубокой детализации с результатами работоспособности функций. Отчет предназначен для использования менеджерским составом проекта для получения общей картины о ситуации с дефектами по программному продукту.

Для каждой новой версии продукта создаются новые колонки для фиксации результатов тестирования (Pass/Fail и Описание).

9.2 Протокол тестирования

Основной целью протоколов тестирования является глубокая детализация описания действий производимых тестировщиком. В документе приведен полный перечень полей, опций, типов данных, вводимых в поля, порядок действий тестировщика, системные проверки и т.д. Документ имеет такой же формат как и журнал тестирования и детально описывает местоположение дефектов в системе. Документ предназначен для использования в основном командой тестирования и разработчиками, для облегчения процесса тестирования и исправления дефектов.

9.3 Отчет о зарегистрированных дефектах

Данный отчет дублирует информацию о зарегистрированных дефектах и служит для удобства заказчика и разработчиков. Документ создается в формате Excel и содержит следующие столбцы:

ID	Состояние	Модуль	Название	Описание	Owner	Важность	Приоритет
№ дефекта	Текущее состояние	Модуль в котором	Краткое описание	Полное описание	Текущий владелец	Важность дефекта	Приоритет исправления

	дефекта	обнаружен дефект	дефекта	дефекта	дефекта		